Day 04

Loops

for · while · break · continue · pass · loop-else

Intermediate Phase · Session 3 of 10 · 2 Hours

Duration	2 hours
Phase	Intermediate (Days 7–16)
Prereqs	Day 2 (Variables), Day 3 (Control Flow)
Next Up	Day 5 — Functions & Scope

What Students Will Learn Today

for loop — Iterate over lists, strings, and any sequence in Python.

range() — Generate number sequences for counting loops.

while loop — Repeat a block of code as long as a condition is True.

break / continue — Exit a loop early or skip the current iteration.

pass — A no-op placeholder for incomplete code blocks.

loop-else — Run a block only if a loop completed without hitting break.

SECTION 1 — SESSION OVERVIEW

Session Goal

By the end of this session students will be able to write **for** and **while** loops to automate repetitive tasks, use **range()** to generate number sequences, control loop flow with **break**, **continue**, and **pass**, and apply the Python-specific **loop-else** pattern for search scenarios.

Minute-by-Minute Timeline

Time	Activity	Detail
0:00 – 0:10	Recap (Day 3)	Quick quiz on if/elif/else and comparison operators
0:10 – 0:30	for loop	Iterating over lists, strings, and range()
0:30 – 0:50	while loop	Condition-driven repetition; infinite loop pitfalls
0:50 – 1:05	break / continue / pass	Flow control inside loops with live demos
1:05 – 1:20	loop-else	Python's unique else clause; search pattern examples
1:20 – 1:45	Guided examples	Four India-context problems solved live
1:45 – 2:00	Wrap-up	Student recap, mini-challenge, preview of Day 5

■ Instructor Note

Demo every loop live — students grasp loops fastest by watching code run. Emphasise that forgetting to update the counter in a while loop causes infinite loops.

■■ Common Pitfall

Students mix up break (exits the loop) and continue (skips to the next iteration). Show both side-by-side on the same example.

SECTION 2 — PREREQUISITES & PREPARATION

Required Knowledge

Day 2	Variables, data types, type casting
Day 3	if/elif/else, comparison + logical operators, match statement

Instructor Pre-Session Checklist

☐	Open VS Code with a blank day04_loops.py file
☐	Have Python REPL ready in the terminal for quick live demos
☐	Verify all students completed and ran Day 3 exercise files
☐	Write range(5), range(2, 7), range(10, 0, -2) on the board
☐	Print / share the day04_loops.py and day04_practise_problems.py files

SECTION 3 — STEP-BY-STEP INSTRUCTION GUIDE

Step 1 of 5 — The for loop (20 min)

for <item> in <sequence>: — iterate over any iterable

What to Say

"A **for** loop walks through every item in a sequence and runs your indented code block once for each one. Python does the counting for you — you just say what to iterate over. Lists, strings, range() — all work the same way."

Live Demo

```
# Loop over a list of cities
cities = ['Mumbai', 'Delhi', 'Pune']
for city in cities:
    print(f'Visiting: {city}')

# Output:
# Visiting: Mumbai
# Visiting: Delhi
# Visiting: Pune
```

Looping over a string

```
for letter in 'Arjun':
    print(letter, end=' ')
# Output: A r j u n
```

■ Done When…

All students can write a for loop that prints each item of a list on its own line.

Step 2 of 5 — range() (10 min)

range(stop) · range(start, stop) · range(start, stop, step)

What to Say

"**range()** creates a sequence of numbers on demand — Python doesn't store all the numbers in memory, it generates them one at a time. It has three forms. Remind students: range() is right-exclusive — range(5) gives 0,1,2,3,4, not 5."

```
range(5)           # 0 1 2 3 4
range(2, 7)        # 2 3 4 5 6
range(10, 0, -2)   # 10 8 6 4 2

for i in range(1, 6):
    print(i, end=' ')
# Output: 1 2 3 4 5
```

■ Tip

Show range(len(my_list)) to loop with index access: students will use this pattern in Day 6 (Data Structures).

Step 3 of 5 — The while loop (20 min)

while <condition>: — repeat until the condition becomes False

What to Say

"A **while** loop is for when you don't know in advance how many iterations you'll need — you just keep going until a condition changes. The most common beginner mistake: forgetting to update the variable inside the loop, which causes an infinite loop. Press Ctrl+C to stop one."

```
balance = 1000   # rupees

while balance > 0:
    balance -= 300          # spend 300 each iteration
    print(f'Balance: Rs {balance}')

# Output:
# Balance: Rs 700
# Balance: Rs 400
# Balance: Rs 100
# Balance: Rs -200
```

■■ Infinite Loop Warning

If students forget 'balance -= 300', the condition is always True. Show Ctrl+C to abort. This is the #1 while loop bug.

Step 4 of 5 — break, continue, pass (15 min)

Control what happens inside each iteration

What to Say

"Three keywords give you fine-grained control inside a loop. **break** exits the loop entirely. **continue** skips the rest of this iteration and moves to the next one. **pass** does absolutely nothing — it's a placeholder so Python doesn't complain about an empty block."

```
# break — exit when found
for n in range(1, 20):
    if n % 7 == 0:
        print(f'First multiple of 7: {n}')
        break
# Output: First multiple of 7: 7

# continue — skip odd numbers
for n in range(1, 11):
    if n % 2 != 0:
        continue
    print(n, end=' ')    # 2 4 6 8 10
```

Error / Symptom	Cause	Fix
Loop runs forever	Missing counter update in while loop	Add n += 1 or balance -= amount inside the loop
Skipping wrong items	continue vs break confused	break = quit loop; continue = next iteration
SyntaxError on pass	pass inside for without any other code	pass is valid; empty block without pass is not

Step 5 of 5 — else on loops (15 min)

else runs ONLY when no break was hit

What to Say

"Python lets you attach an **else** clause to a loop. The else block runs only if the loop completed normally — without ever hitting a break. This is extremely useful for "search and report" patterns. Most other languages don't have this feature."

```
stock = ['Rice', 'Dal', 'Atta']

for item in stock:
    if item == 'Oil':
        print('Oil is in stock!')
        break
else:
    print('Oil not found — add to list.')
# Output: Oil not found — add to list.
```

■ Done When…

Students can explain in their own words: 'The else runs when the loop finishes without a break.'

SECTION 4 — SESSION WRAP-UP

Student Completion Checklist

- ■ Write a for loop that prints every item in a list
- ■ Use range(start, stop, step) to generate a countdown
- ■ Write a while loop that stops when a rupee balance hits zero
- ■ Use break to exit a loop early when a target is found
- ■ Use continue to skip unwanted items in a loop
- ■ Use pass as a placeholder inside an if block inside a loop
- ■ Explain when the loop-else block runs (and when it doesn't)

Key Concepts Recap

for loop	Iterates over any sequence (list, string, range, tuple)
range()	Generates lazy number sequences; right-exclusive
while loop	Repeats while a condition is True; can be infinite if not careful
break	Exits the loop immediately
continue	Skips remaining code in this iteration; goes to next
pass	No-op placeholder; makes empty blocks syntactically valid
loop-else	else block runs only when loop exits without break

Troubleshooting Notes

Error / Symptom	Cause	Fix
Infinite while loop	Counter not updated	Add increment / decrement inside the loop body
for loop skips last item	Off-by-one in range()	Use range(1, n+1) if you want to include n
else runs unexpectedly	loop-else concept misunderstood	Remind: else runs ONLY if no break was hit
NameError inside loop	Variable used before assignment	Define counter / accumulator before the loop

Next Session Preview — Day 5: Functions & Scope

Students will learn to wrap reusable logic inside **def** blocks, pass data in with parameters, return results with **return**, use ***args** and ****kwargs** for flexible signatures, and understand how Python's LEGB scope rule determines variable visibility.

SECTION 5 — QUICK REFERENCE CARD

Loop Cheat Sheet

Syntax / Pattern	Example
`for item in seq:`	`for city in cities: print(city)`
`for i in range(n):`	`for i in range(5): print(i)`
`range(start, stop, step)`	`range(10, 0, -2)  ->  10, 8, 6, 4, 2`
`while condition:`	`while balance > 0: balance -= 100`
`break`	`if found: break`
`continue`	`if n % 2 != 0: continue`
`pass`	`if todo: pass  # placeholder`
`for...else:`	`for x in seq: ... else: print('not found')`
`enumerate(seq)`	`for i, val in enumerate(seq): ...`

VS Code Shortcuts for Today

Ctrl + /	Toggle line comment
Ctrl + D	Select next occurrence of selection
Shift + Alt + Down	Duplicate current line
Ctrl + Enter (REPL)	Run selection in Python Interactive
Ctrl + C (Terminal)	Abort a running / infinite loop